

Data Engineering Workflow Platform

Open Source Data Engineering Platform สำหรับสร้างและจัดการ Data Pipeline ตั้งแต่ Data Source → Data Lake → Transformation → Data Quality → Data Warehouse → Analytics → AI



1. Project Overview

Data Engineering Workflow Platform (DEWP) เป็นแพลตฟอร์มสำหรับสร้าง Data Engineering Workflow แบบครบวงจร โดยเน้นให้สามารถ

- Ingest ข้อมูล
- Transform ข้อมูล
- Validate ข้อมูล
- Orchestrate Pipeline
- Store ข้อมูล
- Monitor Pipeline
- Track Data Lineage
- ส่งข้อมูลต่อให้ BI, ML และ AI

แนวคิดหลัก:

```
DATA
↓
PIPELINE
↓
QUALITY
↓
WAREHOUSE
↓
ANALYTICS
↓
AI
```

2. Quick Start

2.1 Requirements

ติดตั้งเครื่องมือพื้นฐาน:

```
brew install git python uv  
brew install --cask docker
```

ตรวจสอบ:

```
git --version  
python3 --version  
uv --version  
docker --version  
docker compose version
```

3. Create Project

สร้าง Project:

```
mkdir data-engineering-workflow  
cd data-engineering-workflow
```

สร้าง Python Environment:

```
uv venv --python 3.13  
source .venv/bin/activate
```

ติดตั้ง Python Packages:

```
uv pip install \  
    pandas \  
    polars \  
    sqlalchemy \  
    psycpg2-binary \  
    requests \  
    python-dotenv \  
    pydantic \  
    dbt-postgres
```

4. Project Structure

สร้างโครงสร้างเริ่มต้น:

```
mkdir -p \  
dags \  
ingestion \  
transformation \  
quality \  
warehouse \  
dbt/models/staging \  
dbt/models/intermediate \  
dbt/models/marts \  
data/bronze \  
data/silver \  
data/gold \  
tests \  
scripts \  
notebooks \  
docs \  
monitoring
```

โครงสร้าง:

```
data-engineering-workflow/  
|  
|— dags/  
|— ingestion/  
|— transformation/  
|— quality/  
|— warehouse/  
  
|— dbt/  
|   |— models/  
|       |— staging/  
|       |— intermediate/  
|       |— marts/  
  
|— data/  
|   |— bronze/  
|   |— silver/  
|   |— gold/  
  
|— tests/  
|— scripts/  
|— notebooks/  
|— docs/  
|— monitoring/  
  
|— docker-compose.yml  
|— requirements.txt
```

```
|— .env
|— .gitignore
|— README.md
```

5. Start Infrastructure

Development Infrastructure ใช้:

```
Docker
|
|— PostgreSQL
|— MinIO
|— Airflow
```

เริ่มระบบ:

```
docker compose up -d
```

ตรวจสอบ:

```
docker compose ps
```

หยุดระบบ:

```
docker compose down
```

6. First Pipeline

Pipeline ตัวแรกใช้ Sales Dataset:

```
CSV
↓
Extract
↓
Validate
↓
Bronze
```

```
↓
Transform
↓
Silver
↓
Data Quality
↓
Gold
↓
PostgreSQL
```

7. Create Sample Data

สร้างไฟล์:

```
data/bronze/sales.csv
```

ข้อมูลตัวอย่าง:

```
transaction_id,transaction_date,customer_id,product_id,quantity,unit_price
TX001,2026-09-01,C001,P001,2,100.00
TX002,2026-09-01,C002,P002,1,250.00
TX003,2026-09-02,C001,P003,3,150.00
TX004,2026-09-02,C003,P001,5,100.00
TX005,2026-09-03,C004,P002,2,250.00
TX006,2026-09-03,C002,P003,1,150.00
```

8. Run Transformation

ตัวอย่าง Python:

```
python transformation/sales.py
```

ผลลัพธ์:

```
data/
|
├─ bronze/
```

```
└─ sales.csv
└─ silver/
   └─ sales_clean.csv
└─ gold/
   └─ sales_daily.csv
```

9. Verify the Result

ตรวจสอบข้อมูล:

```
cat data/silver/sales_clean.csv
```

ตรวจสอบ Gold:

```
cat data/gold/sales_daily.csv
```

ตัวอย่างผลลัพธ์:

```
transaction_date,transaction_count,units_sold,revenue
2026-09-01,2,3,450
2026-09-02,2,8,750
2026-09-03,2,3,650
```

10. Verify PostgreSQL

เข้า PostgreSQL:

```
docker exec -it de-postgres \
psql -U dataeng -d datawarehouse
```

ตรวจสอบ Schema:

```
\dn
```

ตรวจสอบ Tables:

```
\dt raw.*
\dt staging.*
\dt mart.*
```

Query:

```
SELECT *
FROM mart.sales_daily;
```

11. Architecture

ระบบประกอบด้วย:

```
graph TD
    DS[Data Sources] --> I[Ingestion]
    I --> DL[Data Lake]
    DL --> B[Bronze]
    B --> T[Transformation]
    T --> S[Silver]
    S --> DQ[Data Quality]
    DQ --> G[Gold]
    G --> DW[Data Warehouse]
    DW --> BA[BI / Analytics / ML / AI]
```

flowchart TB

```
SOURCES["DATA SOURCES"]
INGEST["INGESTION"]
LAKE["DATA LAKE MinIO"]
BRONZE["BRONZE Raw"]
TRANSFORM["TRANSFORMATION Python / SQL / dbt / Spark"]
SILVER["SILVER Cleaned"]
```

```
QUALITY["DATA QUALITY"]
GOLD["GOLD Business Ready"]
WAREHOUSE["DATA WAREHOUSE PostgreSQL"]
CONSUMPTION["BI / Analytics / ML / AI"]

SOURCES --> INGEST
INGEST --> LAKE
LAKE --> BRONZE
BRONZE --> TRANSFORM
TRANSFORM --> SILVER
SILVER --> QUALITY
QUALITY --> GOLD
GOLD --> WAREHOUSE
WAREHOUSE --> CONSUMPTION
```

12. Data Workflow

flowchart TD

```
START(["START"])
EXTRACT["Extract"]
VALIDATE["Validate"]
BRONZE["Load Bronze"]
CLEAN["Clean"]
TRANSFORM["Transform"]
SILVER["Load Silver"]
QUALITY["Data Quality"]
GATE{"PASS?"}
QUARANTINE["Quarantine"]
GOLD["Build Gold"]
WAREHOUSE["Load Warehouse"]
PUBLISH["Publish"]
END(["SUCCESS"])
```

```
START --> EXTRACT
EXTRACT --> VALIDATE
VALIDATE --> BRONZE
BRONZE --> CLEAN
CLEAN --> TRANSFORM
TRANSFORM --> SILVER
SILVER --> QUALITY
QUALITY --> GATE
```

```
GATE -->|YES| GOLD
GATE -->|NO| QUARANTINE
```

```
GOLD --> WAREHOUSE
```

```
WAREHOUSE --> PUBLISH
PUBLISH --> END
```

13. Bronze / Silver / Gold

Project ใช้ **Medallion Architecture**

flowchart LR

```
SOURCE["SOURCE"]
BRONZE["BRONZE Raw Data"]
SILVER["SILVER Cleaned Validated"]
GOLD["GOLD Business Ready"]
```

```
SOURCE --> BRONZE
BRONZE --> SILVER
SILVER --> GOLD
```

Bronze

ข้อมูลต้นฉบับ:

```
Raw
Original
Immutable
Traceable
```

Silver

ข้อมูลที่ผ่านการ:

```
Cleaning
Standardization
Deduplication
Validation
```

Gold

ข้อมูลพร้อมสำหรับ:

Analytics
Reporting
BI
ML
AI

14. Data Quality

ตัวอย่าง Quality Rules:

```
transaction_id
  → UNIQUE

customer_id
  → NOT NULL

quantity
  → > 0

unit_price
  → >= 0

transaction_date
  → VALID DATE
```

Architecture:

flowchart TD

```
DATA["Dataset"]
SCHEMA["Schema"]
NULL["NULL"]
DUP["Duplicate"]
TYPE["Data Type"]
RANGE["Range"]
FRESH["Freshness"]
BUSINESS["Business Rule"]
GATE{"Quality Gate"}
PASS["PASS"]
FAIL["FAIL"]
QUARANTINE["Quarantine"]
```

```
DATA --> SCHEMA
SCHEMA --> NULL
NULL --> DUP
```

```
DUP --> TYPE
TYPE --> RANGE
RANGE --> FRESH
FRESH --> BUSINESS
BUSINESS --> GATE

GATE --> |PASS| PASS
GATE --> |FAIL| FAIL

FAIL --> QUARANTINE
```

15. Apache Airflow

Airflow ทำหน้าที่เป็น Workflow Orchestrator

flowchart TD

```
DAG["Airflow DAG"]
INGEST["Ingestion"]
BRONZE["Bronze"]
SILVER["Silver"]
DBT["dbt"]
QUALITY["Quality"]
GOLD["Gold"]
WAREHOUSE["Warehouse"]
MONITOR["Monitoring"]

DAG --> INGEST
INGEST --> BRONZE
BRONZE --> SILVER
SILVER --> DBT
DBT --> QUALITY
QUALITY --> GOLD
GOLD --> WAREHOUSE
WAREHOUSE --> MONITOR
```

Airflow รับผิดชอบ:

- Scheduling
- Task Dependency
- Retry
- Failure Handling
- Monitoring
- Alerting

16. dbt

dbt ใช้สำหรับ SQL Transformation

```
Raw
↓
Staging
↓
Intermediate
↓
Mart
```

โครงสร้าง:

```
dbt/
├── models/
│   ├── staging/
│   │   └── stg_sales.sql
│   ├── intermediate/
│   │   └── int_sales.sql
│   └── marts/
│       └── fct_sales.sql
├── tests/
└── dbt_project.yml
```

คำสั่ง:

```
dbt debug
```

```
dbt run
```

```
dbt test
```

17. Data Lake

```
MinIO
├── bronze
├── silver
└── gold
```

Data Flow:

flowchart LR

```
SOURCE["Data Source"]
BRONZE["MinIO Bronze"]
SILVER["MinIO Silver"]
GOLD["MinIO Gold"]
```

```
SOURCE --> BRONZE
BRONZE --> SILVER
SILVER --> GOLD
```

18. Data Warehouse

PostgreSQL:

```
PostgreSQL
├── raw
├── staging
├── intermediate
└── mart
```

ตัวอย่าง:

```
CREATE SCHEMA IF NOT EXISTS raw;

CREATE SCHEMA IF NOT EXISTS staging;

CREATE SCHEMA IF NOT EXISTS mart;
```

19. Data Lineage

flowchart LR

```
SOURCE["Source"]
BRONZE["Bronze"]
SILVER["Silver"]
GOLD["Gold"]
WAREHOUSE["Warehouse"]
DASHBOARD["Dashboard"]
AI["AI / RAG"]
```

```
SOURCE --> BRONZE
BRONZE --> SILVER
SILVER --> GOLD
GOLD --> WAREHOUSE
WAREHOUSE --> DASHBOARD
WAREHOUSE --> AI
```

สามารถติดตาม:

```
Source
↓
Transformation
↓
Dataset
↓
Warehouse
↓
Dashboard / AI
```

20. Monitoring

Monitoring ควรเก็บ:

```
Pipeline Status
Task Duration
Records Processed
Records Failed
Data Freshness
Quality Score
Error Rate
Retry Count
```

flowchart LR

```
PIPELINE["Pipeline"]
LOGS["Logs"]
METRICS["Metrics"]
ALERT["Alerts"]
DASHBOARD["Monitoring Dashboard"]
```

```
PIPELINE --> LOGS
PIPELINE --> METRICS
```

```
LOGS --> DASHBOARD
METRICS --> DASHBOARD
```

```
METRICS --> ALERT
```

21. Technology Stack

Layer	Technology
Language	Python
Query	SQL
Container	Docker
Orchestration	Apache Airflow
Transformation	dbt
Data Lake	MinIO
Warehouse	PostgreSQL
Processing	Pandas / Polars
Distributed Processing	Apache Spark
Quality	dbt Tests / Great Expectations
Lineage	OpenLineage
Catalog	DataHub
Monitoring	Prometheus / Grafana
Version Control	Git
CI/CD	GitHub Actions

22. Project Structure

```
data-engineering-workflow/  
├── dags/  
├── ingestion/  
├── transformation/  
├── quality/  
├── warehouse/  
  
├── dbt/  
│   ├── models/  
│   │   ├── staging/  
│   │   ├── intermediate/  
│   │   └── marts/  
│   ├── tests/  
│   └── dbt_project.yml  
  
├── data/  
│   ├── bronze/  
│   ├── silver/  
│   └── gold/  
  
├── monitoring/  
├── tests/  
├── notebooks/  
├── scripts/  
├── docs/  
  
├── docker/  
├── .github/  
│   └── workflows/  
  
├── docker-compose.yml  
├── requirements.txt  
├── .env.example  
├── .gitignore  
├── README.md  
└── LICENSE
```

23. Data Model

ตัวอย่าง Sales Data Warehouse:

erDiagram

```

CUSTOMER {
    string customer_id PK
    string customer_name
    string segment
}

PRODUCT {
    string product_id PK
    string product_name
    string category
    decimal price
}

DATE {
    int date_key PK
    date date
    int year
    int month
    int day
}

SALES {
    string transaction_id PK
    string customer_id FK
    string product_id FK
    int date_key FK
    int quantity
    decimal unit_price
    decimal total_amount
}

CUSTOMER ||--o{ SALES : buys
PRODUCT ||--o{ SALES : contains
DATE ||--o{ SALES : occurs

```

24. AI / RAG Integration

Data Engineering Platform สามารถเป็นฐานข้อมูลสำหรับ AI:

flowchart TB

```

DATA["Enterprise Data"]
PLATFORM["Data Engineering Platform"]
LAKE["Data Lake"]
WAREHOUSE["Data Warehouse"]
KNOWLEDGE["Knowledge Layer"]
RAG["RAG"]
AGENT["AI Agent"]

```

DATA --> PLATFORM

PLATFORM --> LAKE

PLATFORM --> WAREHOUSE

LAKE --> KNOWLEDGE

WAREHOUSE --> KNOWLEDGE

KNOWLEDGE --> RAG

RAG --> AGENT

รองรับ:

- Knowledge Base
- RAG
- Embedding Pipeline
- Vector Search
- AI Agent
- Document Processing
- Retrieval Evaluation

25. Distributed Processing

เมื่อข้อมูลมีขนาดใหญ่ สามารถเพิ่ม Apache Spark:

flowchart LR

```
SOURCE["Large Data"]
LAKE["Data Lake"]
SPARK["Apache Spark"]
SILVER["Silver"]
GOLD["Gold"]
WAREHOUSE["Warehouse"]
```

```
SOURCE --> LAKE
LAKE --> SPARK
SPARK --> SILVER
SILVER --> GOLD
GOLD --> WAREHOUSE
```

แนวทางเลือก Processing Engine:

Small Data

↓

Python / Pandas

Medium Data

↓

Polars / SQL

Large Data

↓

Apache Spark

26. Development Roadmap

Phase 01

Foundation

↓

Phase 02

Data Ingestion

↓

Phase 03

Data Lake

↓

Phase 04

ETL / ELT

↓

Phase 05

Airflow

↓

Phase 06

dbt

↓

Phase 07

Data Quality

↓

Phase 08

Data Warehouse

↓

Phase 09

Data Lineage

↓

Phase 10

Data Catalog

↓

Phase 11

Monitoring

↓

Phase 12

CI/CD

↓

Phase 13
Spark
↓
Phase 14
Production
↓
Phase 15
AI / RAG / Agent

27. Production Architecture

flowchart TB

USERS["Users / Applications"]

subgraph SOURCES["DATA SOURCES"]

DB["Databases"]

API["APIs"]

FILE["Files"]

EVENT["Events"]

end

subgraph PLATFORM["DATA ENGINEERING PLATFORM"]

INGEST["Ingestion"]

LAKE["Data Lake"]

BRONZE["Bronze"]

SILVER["Silver"]

QUALITY["Data Quality"]

GOLD["Gold"]

WH["Data Warehouse"]

LINEAGE["Lineage"]

CATALOG["Catalog"]

MONITOR["Monitoring"]

end

AIRFLOW["Apache Airflow"]

subgraph CONSUMPTION["CONSUMPTION"]

BI["BI"]

ANALYTICS["Analytics"]

ML["ML"]

AI["AI / RAG / Agent"]

end

DB --> INGEST

API --> INGEST

FILE --> INGEST

EVENT --> INGEST

INGEST --> LAKE
LAKE --> BRONZE
BRONZE --> SILVER
SILVER --> QUALITY
QUALITY --> GOLD
GOLD --> WH

WH --> BI
WH --> ANALYTICS
WH --> ML
WH --> AI

AIRFLOW -. -> INGEST
AIRFLOW -. -> BRONZE
AIRFLOW -. -> SILVER
AIRFLOW -. -> GOLD
AIRFLOW -. -> QUALITY
AIRFLOW -. -> WH

QUALITY --> LINEAGE
WH --> LINEAGE
LINEAGE --> CATALOG

AIRFLOW -. -> MONITOR
QUALITY -. -> MONITOR
WH -. -> MONITOR

USERS --> CONSUMPTION

28. Production Checklist

Infrastructure

- ☐ Docker
- ☐ PostgreSQL
- ☐ MinIO
- ☐ Airflow

Pipeline

- ☐ Ingestion
- ☐ Bronze
- ☐ Silver
- ☐ Gold
- ☐ Warehouse

Quality

- ☐ Schema Validation
- ☐ NULL Check
- ☐ Duplicate Check
- ☐ Business Rules
- ☐ Freshness

Operations

- ☐ Scheduling
- ☐ Retry
- ☐ Logging
- ☐ Monitoring
- ☐ Alerting

Governance

- ☐ Data Lineage
- ☐ Data Catalog
- ☐ Metadata
- ☐ Data Ownership

Engineering

- ☐ Git
- ☐ Tests
- ☐ CI/CD
- ☐ Docker
- ☐ Documentation

Advanced

- ☐ Spark
- ☐ Distributed Processing
- ☐ ML Pipeline
- ☐ RAG
- ☐ AI Agent

29. Final Vision

flowchart TB

```
DATA["DATA"]  
INGEST["INGEST"]  
STORE["STORE"]  
PROCESS["PROCESS"]
```

```
VALIDATE["VALIDATE"]
ORCHESTRATE["ORCHESTRATE"]
GOVERN["GOVERN"]
MONITOR["MONITOR"]
ANALYTICS["ANALYTICS"]
AI["AI"]
```

```
DATA --> INGEST
INGEST --> STORE
STORE --> PROCESS
PROCESS --> VALIDATE
VALIDATE --> ORCHESTRATE
ORCHESTRATE --> GOVERN
GOVERN --> MONITOR
```

```
MONITOR --> ANALYTICS
ANALYTICS --> AI
```

Vision:

Build → Automate → Validate → Observe → Govern → Scale → AI

Data Engineering Workflow Platform ถูกออกแบบให้เริ่มจาก Pipeline ขนาดเล็กบนเครื่อง Local ได้ทันที และสามารถขยายไปสู่ Data Platform, Distributed Processing และ AI Data Infrastructure ได้ในอนาคต.

30. License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

MIT License

Copyright (c) 2026 Data Engineering Workflow Platform

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.